CENG-222 MIPS Assembly Homework – 2

Yağız Efe YILDIRIM – 202411078

Q1:

```
1  swap:
2      sll $t1, $a1, 2
3      add $t1, $a0, $t1
4      lw $t0, 0($t1)
5      lw $t2, 4($t1)
6      sw $t2, 0($t1)
7      sw $t0, 4($t1)
8      jr $ra
```

Line 1: Start the label swap.

Line 2: sll (shift-left-logical) means shift the bits of the number stored in $a1 by 2. This is necessary for basic instructions because we can't use normal integers with mul or mult. So, we shift the bits of the number in the variable by 2. $a1 has the integer k. That means it is multiplied by 4. Then we store the result in $t1 which is k * 4.

Line 3: We stored k * 4 in $t1 in the previous line. Now we find the 0'th and 1'st element in the array which $a0 stores. $a0 has an address in it and we need to add 4 * k to find v[k] because integers need 4 bytes of space and k is the size of the array. So, we add $t1 and $a0, find the address of v[k] and store the result back in $t1 to use it.

Line 4: Load the first word (integer) v[k] into $t0. 0($t1) means, the integer stored in v[k]'s 0'th index which is the first element.

Line 5: As before, load the second word (integer) in v[k] into $t2.

Line 6: Store the integer in $t2 into v[k]'s index 0 which is the first element.

Line 7: Store the integer in $t0 into v[k]'s index 1 which is the second element. Line 6 and 7 swaps the integers stored in v[k]'s index 0 and 1.

Line 8: jr (jump registers unconditionally) means, jump to statement whose address is stored in $ra. So, we return to where the swap procedure is called.

```
1   sort:
2       addi $sp, $sp, -20
3       sw $ra, 16($sp)
4       sw $s3, 12($sp)
5       sw $s2, 8($sp)
6       sw $s1, 4($sp)
7       sw $s0, 0($sp)
8       move $s2, $a0
9       move $s3, $a1
10      move $s0, $zero
11      for1tst:
12          slt $t0, $s0, $s3
13          beq $t0, $zero, exit1
14          addi $s1, $s0, -1
15      for2tst:
16          slti $t0, $s1, 0
17          bne $t0, $zero, exit2
18          sll $t1, $s1, 2
19          add $t2, $s2, $t1
20          lw $t3, 0($t2)
21          lw $t4, 4($t2)
22          slt $t0, $t4, $t3
23          beq $t0, $zero, exit2
24          move $a0, $s2
25          move $a1, $s1
26          jal swap
27          addi $s1, $s1, -1
28          j for2tst
29      exit2:
30          addi $s0, $s0, 1
31          j for1tst
32      exit1:
33          lw $s0, 0($sp)
34          lw $s1, 4($sp)
35          lw $s2, 8($sp)
36          lw $s3, 12($sp)
37          lw $ra, 16($sp)
38          addi $sp, $sp, 20
39          jr $ra
```

Line 1: Start the label sort.

Line 2: Make 5 registers worth of space on the stack.

Line 3-7: Save $ra, $s3, $s2, $s1 and $s0, respectively, on stack. Because the stack is a LIFO (Last In, First Out) based data structure, the first return is going to be to $s0 and the last is going to be $ra which is the caller procedure calling swap.

Line 8: Move the parameter stored in $a0 into $s2.

Line 9: Move the parameter stored in $a1 into $s3.

Line 10: Set $s0 = 0 (i = 0).

Line 11: Start the label for1tst which is the outer loop.

Line 12: If $s0 >= $s3 (i >= n) set $t0 = 0.

Line 13: If $t0 = 0, jump to label exit1.

Line 14: $s1 = $s0 + (-1) which means j = i – 1 Line 15:

Start the label fort2tst which is the inner loop.

Line 16: If $s1 < 0 (j < 0), set $t0 = 1.

Line 17: If $t0 = 1, jump to label exit2.

Line 18: Like before, multiply $s1 by 4 and store the result in $t1.

Line 19: To find the address of v[j], we set $t2 to the addition of $t1 (j * 4) and the address stored in $s2.

Line 20: Set $t3 to the j'th element of the array.

Line 21: Set $t4 to the (j + 1)'th element of the array.

Line 22: If $t4 >= $t3, set $t0 = 0.

Line 23: If $t0 = 0, jump to label exit2.

Line 24: Set the first parameter $a0 for the swap procedure which is v.

Line 25: Set the second parameter $a1 for the swap procedure which is j.

Line 26: Call the swap procedure.

Line 27: When swap is completed, decrease j by 1.

Line 28: Jump to the inner loop to test the new values.

Line 29: Start the label exit2.

Line 30: Increase i by 1.

Line 31: Jump to the outer loop to test the new values.

Line 32: Start the label exit1.

Line 33-37: Restore $s0, $s1, $s2, $s3 and $ra from stack.

Line 38: Restore stack pointer.

Line 39: Return to calling routine.

Q2:

Swap procedure:

```
swap:
    mul $t1, $a1, 4
    add $t1, $a0, $t1
    lw $t0, 0($t1)
    lw $t2, 4($t1)
    sw $t2, 0($t1)
    sw $t0, 4($t1)
    jr $ra
```

Sort procedure and its labels:

```
sort:
    subi $sp, $sp, 20
    sw $ra, 16($sp)
    sw $s3, 12($sp)
    sw $s2, 8($sp)
    sw $s1, 4($sp)
    sw $s0, 0($sp)
    move $s2, $a0
    move $s3, $a1
    move $s0, $zero
    for1tst:
        bge $s0, $s3, exit1
        subi $s1, $s0, 1
    for2tst:
        blt $s1, 0, exit2
        mul $t1, $s1, 4
        add $t2, $s2, $t1
        lw $t3, 0($t2)
        lw $t4, 4($t2)
        ble $t3, $t4, exit2
        move $a0, $s2
        move $a1, $s1
        jal swap
        subi $s1, $s1, 1
        j for2tst
    exit2:
        addi $s0, $s0, 1
        j for1tst
    exit1:
        lw $s0, 0($sp)
        lw $s1, 4($sp)
        lw $s2, 8($sp)
        lw $s3, 12($sp)
        lw $ra, 16($sp)
        addi $sp, $sp, 20
        jr $ra
```

Q3:

The main routine:

```
.data
array: .word 15, 9, 12, 10, 4 # example array
size:  .word 5                # array size

.text
.globl main
main:
    la $a0, array
    lw $a1, size
    jal sort

    li $v0, 10
    syscall

```

The result of the program:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)
0x10010000	4	9	10	12	15	5

Q4:

As I have provided the codes for both pseudo and basic instructions, let's discuss the topics:

- Readability: Readability of the pseudo-based code is better because of the more understandable layout and instruction set.

- Instruction count: The pseudo-based code has fewer lines because pseudo-instructions simplify the tasks needed.

- The performance is the same because pseudo-instructions run the basic codes when written. They just improve readability and line count. No structural changes either.